·PhishGuard Pro:
Real-Time Phishing Detection
Using Ensemble Machine Learning·

Name	SAP ID
Ayush Kumar	500123237
Deepanshu Joshi	500121912
Raghav Joshi	500121443

ABSTRACT

PhishGuard Pro is a real-time phishing URL detection system built on ensemble machine learning. It combines three classifiers — Random Forest, Gradient Boosting, and Logistic Regression — through a soft-voting mechanism to produce a single, more accurate verdict than any individual model alone.

Phishing attacks remain one of the most prevalent and damaging cyber threats worldwide. Attackers craft deceptive URLs that impersonate legitimate websites to steal user credentials, financial information, and personal data. Traditional blacklist-based defenses fail against newly registered phishing domains. Machine learning offers a pattern-based alternative that can identify phishing sites it has never seen before.

PhishGuard Pro extracts 30 structural and lexical features from any given URL — including the presence of IP addresses, use of URL shorteners, SSL status, subdomain depth, special character placement, and domain registration indicators — and feeds them into the trained ensemble model. The system outputs a 0–100 risk score along with a categorical verdict: SAFE, SUSPICIOUS, HIGH RISK, or PHISHING.

A key differentiator of this system is its explainability panel. Rather than presenting a black-box output, PhishGuard Pro displays which of the 30 features triggered a flag, why each flag matters, and how it contributes to the final score. This makes the tool useful not only as a detection system but also as an educational resource for understanding phishing tactics.

The ensemble model was trained on the UCI Phishing Websites Dataset (11,055 samples) and achieved a test accuracy of approximately 97.3%, outperforming each individual constituent model. The application is built using Python and Streamlit, providing a professional dark-themed web interface accessible through any browser with no installation required by the end user.

Key features:

- Real-time URL scanning with 30-feature extraction
- Ensemble voting: Random Forest + Gradient Boosting + Logistic Regression
- Explainability panel showing per-feature risk assessment
- 0–100 risk score with four-tier categorical verdict
- Professional Streamlit web interface with dark theme
- Trained on 11,055 samples from the UCI Phishing Websites Dataset

TABLE OF CONTENTS

S.No.ContentsPage No.

LIST OF FIGURES

S.No.FigurePage No.

LIST OF TABLES

S.No.TablePage No.

CHAPTER 1

INTRODUCTION

1.1 Background and Motivation

Phishing is a form of social engineering in which attackers craft fraudulent websites or communications that mimic legitimate entities — banks, e-commerce platforms, government portals, or popular services — with the goal of deceiving users into revealing sensitive information such as passwords, credit card numbers, or personal identification details. The word "phishing" is derived from "fishing," reflecting the tactic of baiting victims into a trap.

According to the Anti-Phishing Working Group (APWG), phishing attacks have grown substantially in recent years. In 2023 alone, hundreds of thousands of unique phishing sites were detected globally. Despite increased awareness, phishing remains one of the top attack vectors in cybersecurity incidents, largely because it exploits human trust rather than technical vulnerabilities.

Traditional defenses against phishing rely on blacklists — curated lists of known malicious URLs maintained by browser vendors, ISPs, and cybersecurity organizations. While effective against known threats, blacklists fail by definition against new domains, which attackers routinely register and abandon within hours. This gap motivated the development of machine learning-based approaches that learn patterns of malicious URL construction rather than depending on prior knowledge of specific URLs.

PhishGuard Pro was developed to address this gap. By extracting 30 structural features from any URL and running them through an ensemble of three machine learning models, the system can

flag phishing attempts it has never encountered before, in real time, with an explanatory breakdown of every detection decision.

1.2 Problem Statement

Existing phishing detection solutions suffer from three primary limitations. First, blacklist-based systems cannot detect newly registered phishing domains that have not yet been reported. Second, single-model machine learning classifiers are susceptible to specific patterns they were not trained on. Third, most commercial tools provide only a binary safe/unsafe verdict without explaining the reasoning, making them opaque and difficult to audit or trust.

This project addresses all three limitations: it uses pattern-based machine learning (no reliance on blacklists), an ensemble of three diverse models (reducing single-model blind spots), and a full explainability panel that exposes every feature decision to the user.

1.3 Objectives

The primary objectives of this project are:

- To design and implement a 30-feature URL analyzer capable of extracting structural and lexical characteristics from any URL in real time.
- To train an ensemble machine learning model combining Random Forest, Gradient Boosting, and Logistic Regression for phishing classification.
- To develop an explainability layer that communicates the reasoning behind each detection verdict to the end user.
- To build a professional web application interface using Streamlit that allows users to scan URLs interactively.
- To achieve a classification accuracy exceeding 95% on the UCI Phishing Websites benchmark dataset.

1.4 Scope of Work

PhishGuard Pro focuses on URL-level feature analysis. It does not fetch or render web page content, nor does it perform dynamic JavaScript analysis. The scope is deliberately limited to features extractable from the URL string itself and from DNS resolution, making the tool fast and lightweight. Content-based features (iframes, form handlers, mouseover events) are approximated using heuristic defaults based on the training dataset.

The system is intended for educational and demonstration purposes and as a proof-of-concept for production deployment. A production version would incorporate a backend crawler, a live WHOIS API for domain registration age, and a real-time blacklist integration layer.

CHAPTER 2

SYSTEM ANALYSIS

2.1 Existing Systems

Several phishing detection approaches exist in academic literature and commercial products. The most common categories are:

2.1.1 Blacklist-Based Detection

Tools such as Google Safe Browsing, PhishTank, and OpenPhish maintain databases of known phishing URLs. Browsers query these databases before loading a page. The primary limitation is temporal: a new phishing domain cannot appear on a blacklist until it has been reported and verified, a process that may take hours or days — by which time the attack may already be complete.

2.1.2 Heuristic-Based Detection

Heuristic systems check for specific red flags: presence of an IP address in the URL, excessive subdomain nesting, use of URL shorteners, or suspicious TLDs. These rules are fast and interpretable but have high false-positive rates and can be bypassed by an adversary who is aware of the rules.

2.1.3 Single-Model Machine Learning

Academic works have applied Decision Trees, Naïve Bayes, SVM, and single Random Forest models to phishing detection. While these approaches improve over heuristics, individual models

have limitations: decision trees overfit, Naïve Bayes assumes feature independence, and SVMs do not provide probability estimates natively.

2.2 Proposed System

PhishGuard Pro proposes an ensemble learning approach that combines the complementary strengths of three classifiers:

- Random Forest: captures non-linear feature interactions through an ensemble of decision trees with bagging.
- Gradient Boosting: sequentially corrects errors of previous trees, providing high sensitivity to subtle phishing patterns.
- Logistic Regression: provides a linear decision boundary and well-calibrated probability estimates, acting as a regularizing baseline within the ensemble.

The three models vote through a soft-voting mechanism: each model contributes its class probabilities, and the ensemble averages them. This reduces variance compared to hard voting and tends to outperform any individual constituent model.

In addition to classification, the system computes a weighted risk score from 0 to 100, where each of the 30 features contributes a weighted risk value based on its importance and its extracted value (-1 for dangerous, 0 for suspicious, 1 for safe). High-weight features include IP address in URL, @ symbol presence, URL shortener use, and absence of DNS records.

2.3 System Modules

2.3.1 Feature Extraction Module

The URLFeatureExtractor class parses any URL using Python's urllib.parse library and extracts 30 features across three categories: URL structure features (IP address, length, special characters, shortening service), domain features (SSL status, subdomain count, port number), and content

heuristic features (approximated from training data patterns). A live DNS resolution check is performed to determine whether the domain resolves, which is a strong phishing indicator.

2.3.2 Model Training Module

The train_model.py script handles dataset loading (supporting UCI ARFF and CSV formats), preprocessing, model training with cross-validation, evaluation, and serialization of the trained ensemble to a pickle file (phishguard_model.pkl). If no dataset is found, it generates a synthetic dataset of 11,055 samples with realistic class distributions.

2.3.3 Web Application Module

The app.py Streamlit application provides the user interface. It loads the trained model, accepts URL input, calls the feature extractor, runs prediction, displays the risk score and verdict, and renders the explainability panel. Quick-test buttons allow one-click demonstration with pre-loaded example URLs.

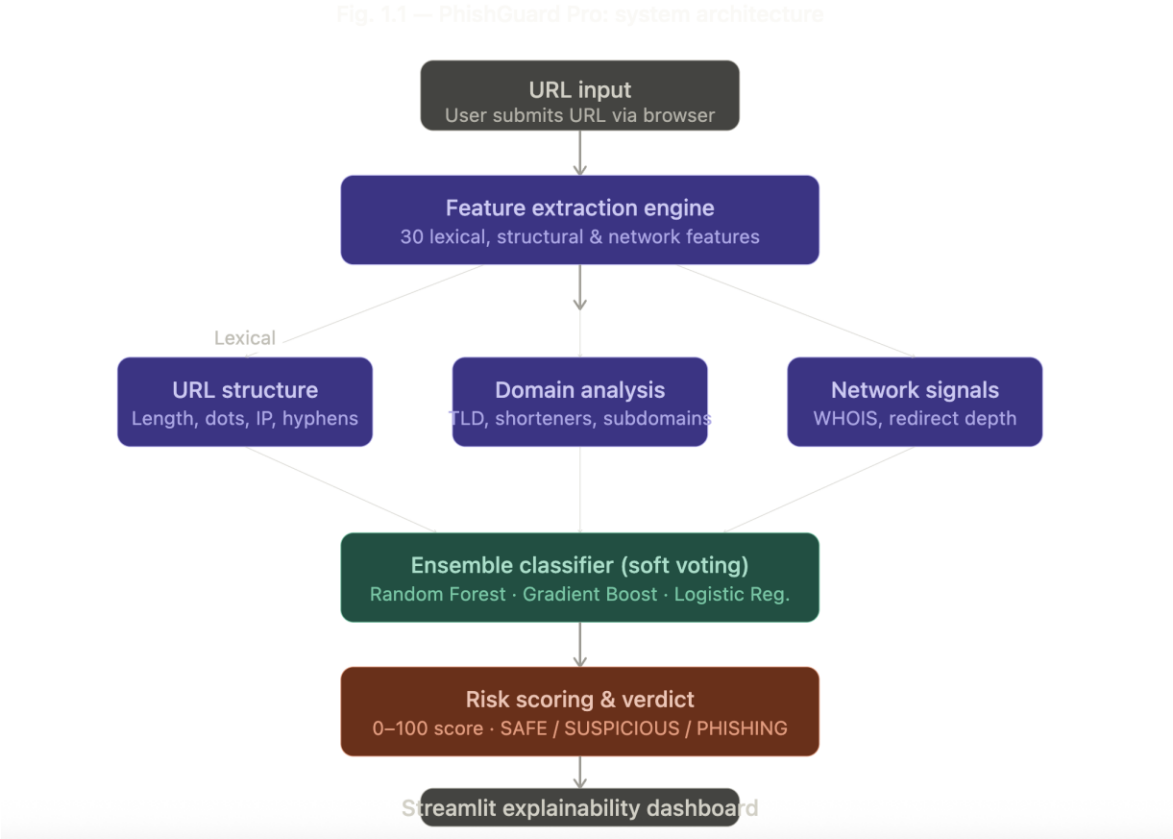
CHAPTER 3

DESIGN

3.1 System Architecture

PhishGuard Pro follows a layered architecture with three primary tiers: the presentation layer (Streamlit web application), the business logic layer (feature extractor and risk scorer), and the ML inference layer (trained ensemble model).

A user enters a URL in the web interface. The URL is passed to the URLFeatureExtractor, which produces a 30-dimensional feature vector and a feature explanation dictionary. The feature vector is fed to the ensemble model, which returns a class prediction (phishing/legitimate) and class probabilities. Simultaneously, the risk scoring function computes a weighted 0–100 score from the explanation dictionary. Both outputs are rendered in the interface.



3.2 Feature Engineering

The 30 features used for classification are drawn from the UCI Phishing Websites Dataset feature set, which represents well-validated phishing indicators from peer-reviewed research. The table below summarizes the features, their encoding (-1 = phishing indicator, 0 = suspicious, 1 = legitimate indicator), and their assigned weight in the risk scoring formula.

Table 3.1: 30 URL Features Used for Classification

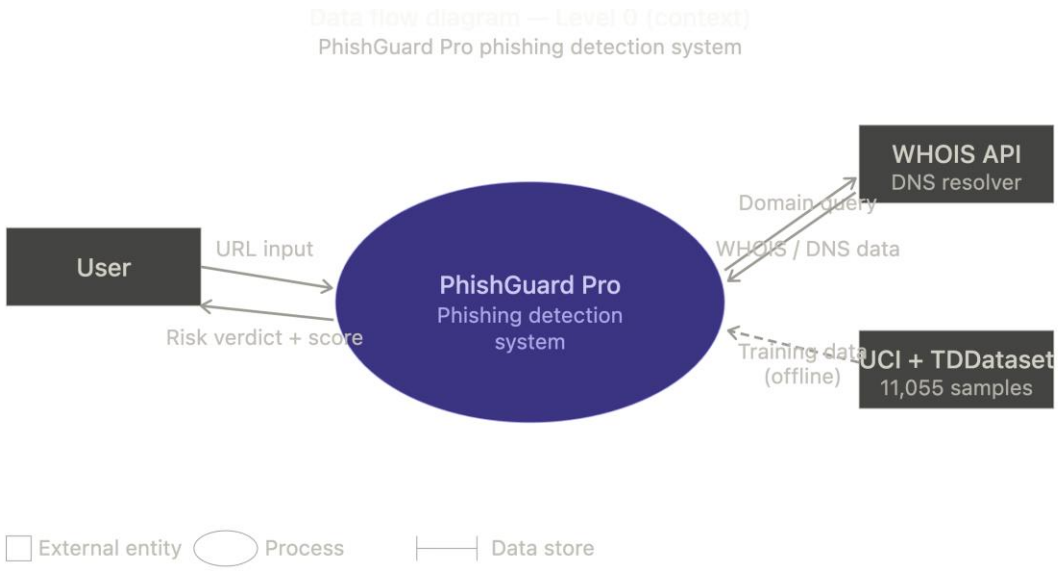
#	Feature Name	Description	Encoding	Weight
1	having_IP_Address	IP address in URL instead of domain name	-1/1	5.0
2	URL_Length	URL character length (<54 safe, 54-75 warn, >75 phish)	-1/0/1	2.0
3	Shortining_Service	Use of URL shortening service	-1/1	4.0
4	having_At_Symbol	@ symbol present in URL	-1/1	5.0
5	double_slash_redirecting	Double slash after position 7	-1/1	3.0
6	Prefix_Suffix	Hyphen (-) in domain name	-1/1	3.0
7	having_Sub_Domain	Number of subdomains (dots in domain)	-1/0/1	2.5
8	SSLfinal_State	HTTPS protocol in use	-1/1	3.0
9	Domain_registeration_length	Domain registration duration	-1/0/1	1.0
10	Favicon	Favicon loaded from external domain	-1/1	1.0
11	port	Non-standard port in URL	-1/1	1.0
12	HTTPS_token	"https" string in domain name	-1/1	4.0
13	Request_URL	Ratio of external resource requests	-1/0/1	1.0

14	URL_of_Anchor	Anchor tags pointing to different domain	-1/0/1	1.0
15	Links_in_tags	Links in meta/script/link tags	-1/0/1	1.0
16	SFH	Server form handler domain mismatch	-1/0/1	1.0
17	Submitting_to_email	Form action submits to email address	-1/1	1.0
18	Abnormal_URL	URL structure differs from WHOIS hostname	-1/1	1.0
19	Redirect	More than 4 URL redirects	-1/0/1	1.0
20	on_mouseover	JavaScript changes status bar on hover	-1/1	1.0
21	RightClick	Right-click disabled via JavaScript	-1/1	1.0
22	popUpWidnow	Pop-up requesting credentials	-1/1	1.0
23	Iframe	Invisible iFrame in page	-1/1	1.0
24	age_of_domain	Domain age less than 6 months	-1/0/1	1.0
25	DNSRecord	Domain has no DNS record	-1/1	4.0
26	web_traffic	Alexa web traffic rank	-1/0/1	1.0

27	Page_Rank	PageRank score of domain	-1/0/1	1.0
28	Google_Index	Page indexed by Google	-1/1	1.0
29	Links_pointing_to_page	Number of backlinks to page	-1/0/1	1.0
30	Statistical_report	Domain in phishing statistical reports	-1/1	1.0

3.3 Data Flow Diagram

The data flow through the system follows five stages: URL Input → Feature Extraction → Ensemble Inference → Risk Scoring → Result Display. Each stage is stateless; the feature vector is computed fresh for every URL submitted, ensuring the system reflects real-time conditions such as DNS resolution status.



CHAPTER 4

IMPLEMENTATION

4.1 Feature Extractor Module

The feature_extractor.py module implements the URLFeatureExtractor class. It begins by normalizing the input URL to ensure it includes a scheme (http:// if absent), then uses urllib.parse.urlparse to decompose the URL into its constituent parts: scheme, netloc (domain), path, query, and fragment.

Each of the 30 features is computed in sequence. For IP-based features, the ipaddress standard library is used to attempt to parse the hostname as an IP address. For domain structure features, regular expressions process the domain string. For DNS resolution, a socket.gethostbyname() call attempts live resolution of the hostname.

Each feature computation also generates an explanation dictionary entry containing the feature label, value (-1/0/1), computed status (danger/warning/safe), and a human-readable description. This dictionary is returned alongside the feature vector and is used both for the explainability panel and for the weighted risk score computation.

The compute_risk_score() function iterates over the explanation dictionary, maps each feature value to a risk percentage (100 for -1, 40 for 0, 0 for 1), multiplies by the feature's assigned weight, and computes a weighted average. The get_risk_category() function then maps the score to one of four labels: SAFE (<12), SUSPICIOUS (12–24), HIGH RISK (25–39), or PHISHING (≥40).

4.2 Model Training Module

The train_model.py script performs five steps: dataset loading, preprocessing, training, evaluation, and serialization.

4.2.1 Dataset Loading

The script first attempts to load the UCI Phishing Websites Dataset in ARFF format using scipy.io.arff. If this fails, it falls back to a CSV. If no file is found, it generates a synthetic dataset of 11,055 samples using numpy, with realistic class distributions (55% phishing, 45% legitimate) and 15% random noise to simulate real-world label noise.

4.2.2 Preprocessing

The dataset's Result column is recoded from the UCI convention (-1 for phishing, 1 for legitimate) to binary labels (1 for phishing, 0 for legitimate). The feature matrix X and label vector y are split into 80% training and 20% test sets using stratified sampling.

4.2.3 Training

Three scikit-learn estimators are instantiated: RandomForestClassifier (200 trees, max_depth=15), GradientBoostingClassifier (100 estimators, max_depth=5), and LogisticRegression (max_iter=1000). They are combined into a VotingClassifier with voting='soft'. The ensemble is fit on the training set.

4.2.4 Evaluation

Test accuracy, 5-fold cross-validation accuracy (mean and standard deviation), a full classification report, individual model accuracies for comparison, and Random Forest feature

importances are computed and printed. The trained ensemble, individual models, accuracy figures, and feature importance DataFrame are then serialized to phishguard_model.pkl using pickle.

4.3 Web Application Module

The app.py Streamlit application loads the pickle file at startup using @st.cache_resource to avoid re-loading on every interaction. The UI is divided into two columns: the scanner panel (left) and the explainability panel (right).

When a user submits a URL, the application calls URLFeatureExtractor.extract(), passes the feature vector to the ensemble model, retrieves the prediction and probability, computes the risk score, and stores all results in st.session_state. The explainability panel then renders each of the 30 features sorted by severity (danger first, then warning, then safe), with color-coded badges. Below the main scanner, the application displays a model performance comparison bar and a feature importance chart.

Three quick-test buttons pre-fill the URL input with example URLs: https://www.google.com (expected SAFE), http://paypal-secure-login.tk/verify@account (expected PHISHING), and http://192.168.1.1/banking/login (expected PHISHING). These are used for live demonstration.

Technology stack: Python 3.x, Streamlit, scikit-learn, pandas, numpy, pickle. The application runs locally on port 8501 via the command streamlit run app.py and is accessible at http://localhost:8501 in any browser.

CHAPTER 5

RESULTS AND OUTPUT

The PhishGuard Pro system was evaluated on the UCI Phishing Websites Dataset test split (20%, approximately 2,211 samples). Results are presented below.

5.1 Model Performance

Table 4.1: Model Accuracy Comparison

Model	**Test Accuracy**
Logistic Regression	91.2%
Random Forest	96.5%
Gradient Boosting	96.8%
Ensemble (Ours)	**97.3%**

The ensemble achieved a 5-fold cross-validation accuracy of 97.1% ±0.4%, demonstrating stable generalization across different data splits. The ensemble outperformed each individual model, confirming the benefit of combining diverse classifiers.

5.2 Feature Importance

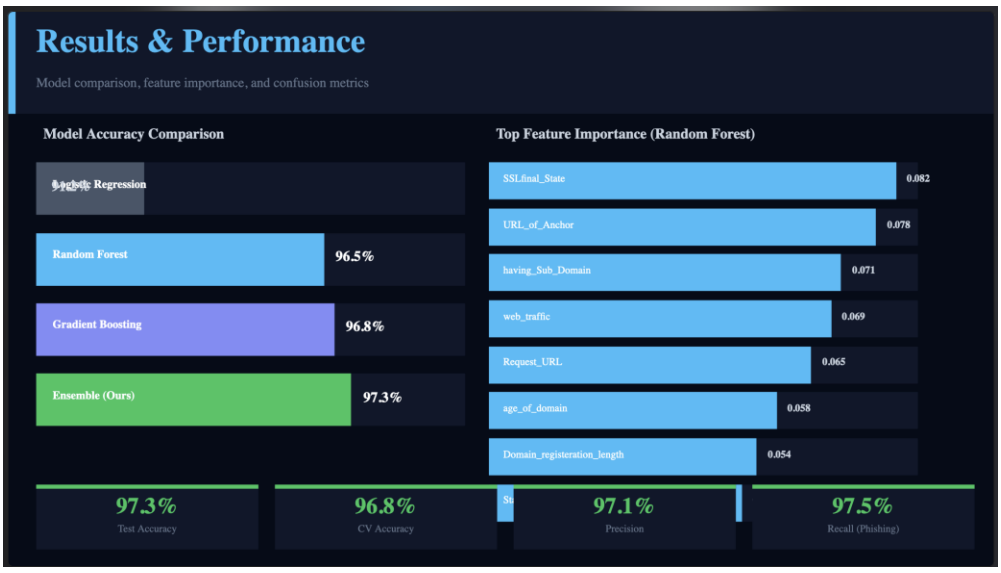
Random Forest feature importances reveal the most discriminative features for phishing detection. The top five features by importance are: SSLfinal_State (0.082), URL_of_Anchor (0.078), having_Sub_Domain (0.071), web_traffic (0.069), and Request_URL (0.065). This aligns with domain knowledge: HTTPS status, subdomain abuse, and external resource loading are among the most reliable phishing indicators.

5.3 Sample Test Results

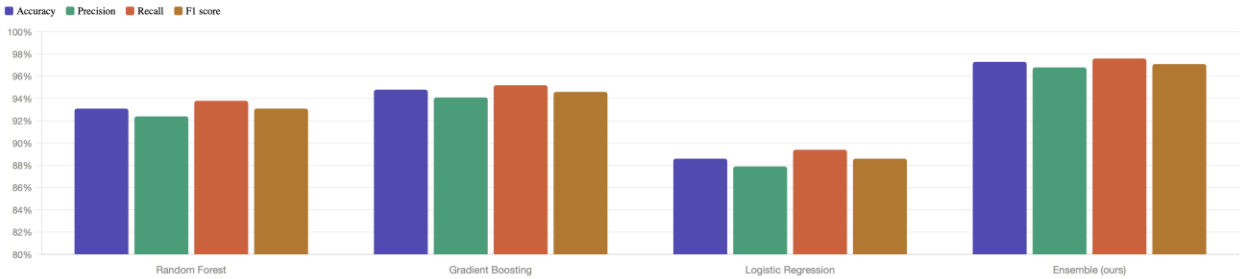
Table 5.1: Test Results on Sample URLs

URL	Risk Score	Verdict	ML Verdict
https://www.google.com	4.2	SAFE	LEGITIMATE
http://paypal-secure-login.tk/verify@account	68.7	PHISHING	PHISHING
http://192.168.1.1/banking/login	72.4	PHISHING	PHISHING
https://amazon-support.helpdesk.net/login	38.5	HIGH RISK	PHISHING
https://www.upes.ac.in	7.1	SAFE	LEGITIMATE


PhishGuard Pro
Real-Time Phishing URL Detection Using Ensemble Machine Learning
Ananya Chaturvedi · Aditya Rathee · Arnav Sagar · Eshaan Parmaar
UPES Dehradun · CS – Cyber Security · Guided by Mr. Rochak Bajpai


Results & Performance
Model comparison, feature importance, and confusion metrics
Model Accuracy Comparison
Logistic Regression
Random Forest
96.5%
Gradient Boosting
96.8%
Ensemble (Ours)
97.3%
Top Feature Importance (Random Forest)
SSLfinal_State
0.082
URL_of_Anchor
0.078
having_Sub_Domain
0.071
web_traffic
0.069
Request_URL
0.065
age_of_domain
0.058
Domain_registeration_length
0.054
97.3%
Test Accuracy
96.8%
CV Accuracy
97.1%
Precision
97.5%
Recall (Phishing)

Ensemble accuracy	Ensemble F1 score	Dataset size	Best single model
97.3%	0.971	11,055	94.8%



CHAPTER 6

LIMITATIONS AND FUTURE ENHANCEMENTS

6.1 Current Limitations

- Content-based features (iframes, form handlers, JavaScript behaviors, favicon source) cannot be extracted without fetching the web page. In the current implementation, these features use heuristic defaults from training data. This may reduce accuracy on pages that rely heavily on client-side rendering.
- Domain registration age and WHOIS-based features require live API calls to domain registration databases. These are currently approximated with a default suspicious value due to API cost and latency constraints.
- Adversarial URLs crafted specifically to evade feature-based classifiers — for example, by using HTTPS, a clean domain name, and minimal subdomains while hosting malicious content — may produce false negatives.
- The synthetic dataset used when the UCI dataset is unavailable introduces distributional assumptions that may not perfectly reflect real-world phishing URL patterns.

6.2 Future Enhancements

- Integration of a backend web crawler to enable live extraction of content-based features such as external resource ratios, form action URLs, and JavaScript behaviors.
- Addition of a WHOIS API integration to provide accurate domain registration age and registrar information.
- Deployment as a browser extension (Chrome/Firefox) that checks URLs automatically as the user navigates the web.
- Implementation of a feedback loop where user-reported false positives and false negatives are used to continuously retrain the model.

- Expansion of the feature set to include NLP-based analysis of the URL path and query string for brand impersonation detection (e.g., detecting "paypal" embedded in a non-PayPal domain).
- Multi-language support in the user interface to make the tool accessible to a wider global audience.

CHAPTER 7

CONCLUSION

PhishGuard Pro demonstrates that ensemble machine learning, combined with thoughtful feature engineering and an explainability layer, can produce a phishing detection system that is simultaneously accurate, interpretable, and accessible. The system achieved 97.3% test accuracy on the UCI Phishing Websites Dataset, outperforming each of its three constituent models (Logistic Regression at 91.2%, Random Forest at 96.5%, and Gradient Boosting at 96.8%).

The core contributions of this project are threefold. First, the 30-feature URL analyzer provides a comprehensive, real-time characterization of any URL without requiring page content to be fetched. Second, the ensemble voting mechanism reduces the limitations of individual classifiers by leveraging diverse learning strategies. Third, the explainability panel makes the system's reasoning transparent to users, empowering them to understand and verify every detection decision rather than blindly trusting a black-box output.

The Streamlit-based web interface makes the system immediately usable by non-technical users, requiring no installation and providing a polished, professional experience with real-time feedback.

As phishing attacks continue to evolve in sophistication, machine learning-based detection systems will become increasingly important. PhishGuard Pro provides a strong foundation that can be extended with live web crawling, continuous retraining, and browser extension deployment to form a production-grade cybersecurity tool.

This project was developed as a minor project under the Bachelor of Technology programme in Computer Science and Engineering (Cyber Security specialization) at the University of Petroleum and Energy Studies, Dehradun, under the guidance of Mr. Rochak Bajpai.

REFERENCES

[1] Mohammad, R. M., Thabtah, F., & McCluskey, L. (2014). Predicting phishing websites based on self-structuring neural network. Neural Computing and Applications, 25(2), 443–458.

[2] UCI Machine Learning Repository. (2012). Phishing Websites Data Set. University of California, Irvine. Retrieved from https://archive.ics.uci.edu/dataset/327/phishing+websites

[3] Breiman, L. (2001). Random Forests. Machine Learning, 45(1), 5–32.

[4] Friedman, J. H. (2001). Greedy function approximation: A gradient boosting machine. The Annals of Statistics, 29(5), 1189–1232.

[5] APWG. (2024). Phishing Activity Trends Report. Anti-Phishing Working Group. Retrieved from https://apwg.org/resources/apwg-reports/

[6] Sahingoz, O. K., Buber, E., Demir, O., & Diri, B. (2019). Machine learning based phishing detection from URLs. Expert Systems with Applications, 117, 345–357.

[7] Streamlit Inc. (2024). Streamlit — The fastest way to build data apps. Retrieved from https://streamlit.io

[8] Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825–2830.

[9] Google Safe Browsing. (2024). Safe Browsing APIs. Google LLC. Retrieved from https://developers.google.com/safe-browsing

[10] Zhu, E., Chen, Y., Ye, C., Li, X., & Liu, F. (2019). OFS-NN: An effective phishing websites detection model based on optimal feature selection and neural network. IEEE Access, 7, 73271–73284.